

PLAN DE TRABAJO RETO 1

INFORMÁTICA 2

2026

Revisión 1

Juan Pablo Rincón
11/09/2026

Índice

1. Introducción	2
2. Diseño del Juego	2
2.1. Los tres casos de distribución en bytes	2
2.2. Tamaño en memoria	2
2.3. Tentativa de módulos	2
2.4. Funciones tentativas	3
3. Conclusiones	3

1. Introducción

En este informe se presenta el desarrollo del proyecto el cual consiste en el juego **Sweet Crush**, aplicando operadores a nivel de bits y la gestión de memoria dinámica.

Cada ficha deberá tener un total de 3 bits empacados de forma continua en un arreglo de unsigned char. La memoria dinámica debe constar en reservar y liberar la memoria según las dimensiones actuales. Por ejemplo, la idea es reservar la memoria inicial del tablero.

Por legibilidad usaremos solo la capacidad horizontal de la terminal; de manera vertical se hará de cuantas filas el usuario lo considere. Tener en cuenta que la idea es no sobredimensionar, solo redimensionar cuando debemos agregar o quitar filas o columnas.

La lógica del juego debe constar en: - Detectar combinaciones. - Resolver si las fichas son simultáneas tanto vertical como horizontalmente. - Detectar las cascadas. - Aplicar desplazamiento por "gravedad". - Rellenar fichas en espacios vacíos y cascadas.

Restricciones del proyecto: - No se podrán usar struct, class, template. - Solo se permite el uso de arreglos y punteros. - No se usarán string; el texto en pantalla deberá hacerse con punteros a char o arreglos char, en su defecto usando cout « con literales. - La reasignación de memoria se hará al reducir filas y columnas SÓLO cuando el uso caiga por debajo del 65% del tamaño reservado actual.

»2. Diseño del Juego

»El juego se compone de una cuadrícula de 3 fichas diferenciables. Por medio de la programación modular (.h y .cpp), el objetivo será implementar funciones que faciliten la estructura del juego y hagan énfasis en las características del código: correctitud, eficiencia y diseño.

»Ejemplo de codificación de fichas:

»

Ficha A = 000, Ficha B = 001, Ficha C = 010 ...

»2.1. Los tres casos de distribución en bytes

»1. Ficha completa en un byte: tamaño-bit <= 5 (quedan al menos 3 bits libres en ese byte). 2. Ficha repartida 2+1: empiezan 2 bits al final del byte N y el tercero al inicio del byte N+1. 3. Ficha repartida 1+2: empieza 1 bit al final del byte N y los otros dos al inicio del byte N+1.

»2.2. Tamaño en memoria

»

$$\text{bits totales} = 3 \cdot \text{filas} \cdot \text{columnas}$$

»

$$\text{bytes totales} = \frac{(\text{bits totales} + 7)}{8}$$

»2.3. Tentativa de módulos

»- Main.cpp: interacción con el usuario y llamado de todas las funciones.
- Manejo-de-bits.h / Manejo-de-bits.cpp: manipulación de bits. - tablero.h

/ tablero.cpp: creación, redimensión y acceso lógico al tablero. - juego.h
/ juego.cpp: combinaciones, cascadas, puntaje y movimientos. - impresion-pantalla.h
/ impresion-pantalla.cpp: todo lo que se visualizará en pantalla (terminal).

»2.4. Funciones tentativas

- »■ leerficha()
- »■ escribirficha()
- »■ bytesnecesarios()
- »■ creartablero()
- »■ destruirtablero()
- »■ Agregarfila()
- »■ Eliminarfila()
- »■ AgregarColumna()
- »■ EliminarColumna()
- »■ DetectarCombinaciones()
- »■ EliminarCombinaciones()
- »■ DetectarCascadas()
- »■ EliminacionFicha()
- »■ Mostrartablero()
- »■ Mostrarestado()

»3. Conclusiones

»El proyecto **Sweet Crush** busca fortalecer el uso de arrays, procesamiento a nivel de bits, uso de punteros, memoria dinámica y funciones.